

claude-windows / d4cf3fd6-b5bd-4ed1-9b57-88eea69333cd

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\d4cf3fd6-b5bd-4ed1-9b57-88eea69333cd\subagents\workflows\wf_8c732492-138\agent-a47789bf8f991c707.jsonl`
- SHA-256: `77548bb9dac8bd24970cb9ac080c7fcce4f9a305a3f598fe61a68a8f2e440e17`
- Source modified: `2026-07-04T08:53:12+00:00`
- Imported at: `2026-07-05T16:48:23+00:00`
- project: `wf_8c732492-138`
- session_id: `d4cf3fd6-b5bd-4ed1-9b57-88eea69333cd`
```

Transcript

1. user (2026-07-04T08:47:59.441Z)

You are fixing a code-review blocker on PR #467 (branch fix/upload-gcs-staging) of Khelsutra/badminton-highlight-indexer.
Your worktree: C:/Users/avidu/Projects/khelsutra-guru/wt-upload-staging (must be on fix/upload-gcs-staging at d3a9629)

ENVIRONMENT (read carefully):

- Windows 11 box; use the PowerShell tool for shell commands (or Bash for POSIX one-liners). Global Python 3.13.13 has the full offline dep set (pytest 9.0.3 works).
- Repo: Khelsutra/badminton-highlight-indexer. You work in an EXISTING git worktree (path given below) that is already checked out on the PR branch at the PR head. Other sibling worktrees exist for other PRs ? do NOT touch them, do NOT touch the main checkout at C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer.
- FIRST: run "git branch --show-current" and "git log --oneline -1" in your worktree and confirm they match the branch+head stated below. If they do not match, STOP and report.
- Do NOT commit. Do NOT push. Do NOT run any gh/network write command. Leave your changes UNCOMMITTED in the worktree; the orchestrator reviews, commits, and pushes.
- MANDATORY verification gates before you report done (run from the worktree root, report exact results):
 1. FULL suite: python -m pytest tests/ -q -p no:cacheprovider (~2-5 min; NEVER run just the changed test file as your final gate ? the whole suite must be green; report exact "N passed, M skipped" counts and any failures verbatim)
 2. Lint: python -m ruff check . (CI pins ruff==0.15.16; if ruff is missing or a different major, python -m pip install ruff==0.15.16)
 3. Types: python -m mypy backend training (CI pins mypy==2.1.0; install if missing)
- Note: sibling agents run their own pytest suites concurrently on this box ? expect slower wall-clock, and use -p no:cacheprovider exactly as given.
- CODE STYLE: match the repo ? rich docstrings that explain WHY and cite the design constraint (see claim_due_job's existing docstring for tone); comments only where the code can't say it; small functions. backend/main.py has an enforced line budget (tests/test_main_module_budget.py) ? avoid growing it.
- Your final message is consumed by the orchestrator via the structured schema. Report HONESTLY ? if a gate is red or you could not finish, say so in the schema fields; never claim green that you didn't see.

REVIEWER BLOCKER (verbatim, from PR #467):

"I found one more config-shape blocker in the staging gate.

The new local-upload staging path decides whether staging is possible with:

```
input_root = storage_cfg.input_root
input_backend = storage_cfg.input_backend or 'local'
if not input_root or input_backend == 'local':
    return _failed(...)
```

That rejects a valid config where storage.input_root is itself an explicit cloud URI and input_backend is left at its default. The storage layer already treats an explicit root like gs://in-bucket/videos as backend-bearing (resolve_input_uri joins under input_root before

consulting input_backend), and the model docs call out input_root='gs://bucket/videos' + 'x.mp4'
=> 'gs://bucket/videos/x.mp4'.

I reproduced on this head with:

```
{ "deployment": { "compute_target": "cloud_run", "storage": { "backend": "gcs", "input_root":  
"gs://in-bucket/videos", "output_root": "gs://out-bucket/results" } }
```

A local upload still fails pre-dispatch with 'needs a cloud input ... or a cloud
storage.input_root' because input_backend remains the default local, even though input_root is
already a gs:// destination.

Suggested fix: classify the configured input_root itself (e.g.
storage.parse_uri(input_root).backend in {'gcs', 's3', 'gdrive-api'} / at least not 'local')
instead of relying on input_backend, and add a regression test where input_root='gs://...' is
set without input_backend='gcs'."

TASK: Fix the config-shape bug in the local-upload staging gate.

- File: backend/orchestration.py, the staging gate at ~lines 253-266 (inside the
input_ref.backend == 'local' branch): the predicate "if not input_root or input_backend ==
'local'" wrongly requires input_backend to be explicitly non-local.
- BEFORE choosing the predicate, study backend/storage/__init__.py: parse_uri(),
resolve_input_uri(), resolve_input_ref() ? understand exactly how input_root + input_backend
combine. CRITICAL downstream consistency: the staging code then builds staged_uri =
f"{input_root.rstrip('/')}/{video_id}/{basename}" and calls storage.put(local_src, staged_uri,
config=...) ? and put() routes by parse_uri(staged_uri).backend. So staging GENUINELY works only
when the staged URI parses to a non-local backend. Consider whether the correct gate is
"parse_uri(input_root).backend != 'local'" (reviewer's suggestion), and whether the legacy shape
(bare/local-looking input_root + explicit non-local input_backend, which the OLD gate accepted)
can even work through put() ? if that legacy shape is broken downstream, prefer classifying the
actual staged destination and make the gate truthful; explain your choice in the PR-reply notes.
Aim for the MINIMAL fix that makes the reviewer's exact repro config work and keeps every
existing test green.
- Keep/adjust the _failed error message so it stays accurate about what config enables staging.
- REGRESSION TEST (required by reviewer): in tests/test_api_cloud_dispatch.py (where the
existing staging tests live ? find the test that asserts staging happens and mimic its fixture
pattern): a config with input_root='gs://in-bucket/videos' and input_backend left at its DEFAULT
(not set / 'local'), compute_target=cloud_run, a local upload input -> staging must proceed
(assert storage.put is called with the gs:// staged URI and the dispatch continues), NOT fail
with the 'needs a cloud input' message. Keep the existing negative test (no input_root at all ->
fail) green.

When done, fill the structured output schema exactly and honestly.

2. user (2026-07-04T08:48:06.110Z)

fix/upload-gcs-staging

d3a9629 review: thread active storage config into the staging put (#461)

3. user (2026-07-04T08:48:10.645Z)

```
200         "status": "failed",  
201         "message": msg,  
202         "results": [r.model_dump() for r in val_results],  
203         "play_segments": [],  
204         "compute": {  
205             "path": "cloud_run" if ran else "not_run",  
206             "requested": req.compute,  
207             "target": "cloud_run",  
208             "dispatched": ran,  
209         },  
210     },  
211     ran,  
212 )  
213  
214     # SPA compute-picker (item 3): a per-request choice overrides the server's  
configured default.  
215     # "local" forces the in-process segmenter; "cloud" forces a Cloud Run dispatch  
(guarded by
```

```

216         # _cloud_available so we fail clearly rather than silently running local); None /
unknown ? default.
217         choice = (req.compute or "").strip().lower()
218         if choice and choice not in ("local", "cloud"):
219             logger.warning(
220                 "[API] ignoring unknown compute=%r (expected 'local'/'cloud'); using server
default.",
221                 req.compute,
222             )
223             choice = ""
224             if choice == "local":
225                 return None # explicit "run on my machine" ? in-process regardless of the
server's target
226             if choice == "cloud":
227                 if not _cloud_available(cfg):
228                     return _failed(
229                         "cloud-serving was requested but this server isn't configured for it "
230                         "(no Cloud Run target). Pick 'run on my machine', or configure cloud
serving.",
231                         False,
232                     )
233                 compute = get_compute_backend(cfg, target_override="cloud_run")
234             else:
235                 compute = get_compute_backend(
236                     cfg
237                 ) # server default (default-OFF unless cloud_run)
238             if compute is None:
239                 return None # default-OFF ? run the in-process segmenter below (unchanged)
240
241             storage_cfg = getattr(cfg, "storage", None)
242
243             # The Cloud Run job fetches the input FROM THE BUCKET ? it can't read a path local
to this box.
244             try:
245                 input_ref = storage.resolve_input_ref(req.video_path, storage_cfg)
246             except ValueError as e:
247                 return _failed(
248                     f"cloud-serving: unresolvable input '{req.video_path}': {e}", False
249                 )
250             if input_ref.backend == "local":
251                 # A browser upload lands on THIS box's local disk; the Cloud Run job (a
different machine)
252                 # can't read it. Stage it UP to the input bucket first, then dispatch from there
(#461). Only
253                 # possible when a CLOUD input_root is configured; without one we genuinely can't
stage ? fail.
254                 input_root = (getattr(storage_cfg, "input_root", "") or "").strip()
255                 input_backend = (
256                     getattr(storage_cfg, "input_backend", "local") or "local"
257                 ).strip()
258                 if not input_root or input_backend == "local":
259                     return _failed(
260                         "cloud-serving needs a cloud input (e.g. gs://?), or a cloud
storage.input_root to "
261                         "stage local uploads into; a path local to this machine can't be read by
the Cloud "
262                         "Run job.",
263                         False,
264                     )
265                 local_src = input_ref.key # resolved local path of the upload
266                 staged_uri =
f"{input_root.rstrip('/')}/{video_id}/{os.path.basename(local_src)}"
267                 try:
268                     notify("staging upload to bucket")
269                     # Thread the active storage settings so backend-specific config survives (S3

```

```

endpoint_url/
270         # region/addressing for R2·D0·MinIO; GCS project/client) ? not a fresh
default client.
271         storage.put(
272             local_src,
273             staged_uri,
274             config=(
275                 storage_cfg.model_dump() if storage_cfg is not None else None
276             ),
277         )
278     except Exception as e: # noqa: BLE001 ? a staging failure is a clean job
failure, not a 500
279         logger.error(
280             "[API] cloud-serving: failed to stage %s -> %s: %s",
281             local_src,
282             staged_uri,
283             e,
284             exc_info=True,
285         )
286         return _failed(
287             f"cloud-serving: failed to stage the uploaded file to {input_root}:
{e}",
288             False,
289         )
290     # Repoint BOTH the dispatch and the DB video record at the staged bucket object:
the L4 job
291     # DETECTs from it, and a later /api/compile re-fetches the same source (the
local upload is
292     # gone once this instance recycles). add_video is an UPSERT ? it only updates
filepath.
293     req.video_path = staged_uri
294     input_ref = storage.resolve_input_ref(staged_uri, storage_cfg)
295     db.add_video(
296         video_id, staged_uri, "processed", [r.model_dump() for r in val_results]
297     )
298     out_root = (getattr(storage_cfg, "output_root", "") or "").strip()
299     if not out_root:
300         return _failed(
301             "cloud-serving needs storage.output_root set to a bucket (e.g. gs://your-
bucket) "
302             "? that's where the Cloud Run job writes the result.",
303             False,
304         )
305
306     notify("dispatching (cloud_run)")
307     logger.info(
308         "[API] cloud-serving: dispatching %s -> Cloud Run Job (out=%s)",
309         req.video_path,
310         out_root,
311     )
312     # Forward the engine's AI-handoff stance so the cloud job runs the SAME detection:
with
313     # skip_ai_handoff=True the WASB windows become rallies directly (no Gemini key
needed); without
314     # forwarding the cloud job would default to the handoff and yield 0 rallies when it
has no key.
315     # (player_pool / max_frames are still dropped on the cloud path ? a separate
documented follow-up.)
316     overrides = []
317     sk = getattr(getattr(cfg, "indexing", None), "skip_ai_handoff", None)
318     if sk is not None:
319         overrides.append(f"indexing.skip_ai_handoff={'true' if sk else 'false'}")

```

4. user (2026-07-04T08:48:10.963Z)

```

1      """Pluggable storage layer: local (default) + S3/GCS first-class buckets + Google Drive.
2
3      URI-driven (see ``ref.parse_uri``). The package import is cheap ? cloud SDKs are
imported
4      lazily, only when a non-local bucket URI is actually used (install the ``storage-s3`` /
5      ``storage-gcs`` extras). The default ``local`` backend needs no third-party deps and
preserves
6      today's behaviour byte-for-byte; ``gdrive://`` likewise needs none ? it reads a locally
MOUNTED
7      Google Drive (Google Drive for Desktop) as plain files.
8
9      Worker/dispatcher entry points speak URIs via the thin helpers ``fetch`` / ``put`` /
10     ``list_uris`` / ``get_storage``; ``local://`` (and bare paths) make ``fetch`` an
identity
11     passthrough. See ``docs/DECOUPLED_COMPUTE``.
12     """
13
14     from __future__ import annotations
15
16     import os
17     import shutil
18     import tempfile
19     from typing import Any, Dict, Iterator, Optional
20
21     from backend.storage.base import StorageBackend
22     from backend.storage.ref import StorageRef, StorageStat, parse_uri, resolve_input_uri
23
24     __all__ = [
25         "StorageBackend",
26         "StorageRef",
27         "StorageStat",
28         "parse_uri",
29         "resolve_input_uri",
30         "get_storage",
31         "fetch",
32         "put",
33         "list_uris",
34         "acquire_local_input",
35         "cleanup_acquired_local_input",
36         "input_is_local",
37         "resolve_input_ref",
38     ]
39
40
41     def _storage_dict(storage_config: Any) -> Dict[str, Any]:
42         """Coerce a StorageConfig (Pydantic) / dict / None into a plain settings dict."""
43         if storage_config is None:
44             return {}
45         if hasattr(storage_config, "model_dump"):
46             return storage_config.model_dump() # type: ignore[no-any-return]
47         return dict(storage_config)
48
49
50     def resolve_input_ref(raw: str, storage_config: Any = None) -> StorageRef:
51         """Resolve a user-supplied input (a path or storage URI) into a ``StorageRef`` under
the input
52         config (``input_backend``/``input_root``). Raises ``ValueError`` for an unsupported
URI scheme ?
53         callers at the API/CLI boundary catch that and return a clean client error (never a
500).
54
55         Use ``ref.backend`` to branch local-vs-remote and ``ref.key`` for the resolved LOCAL
path (e.g.
56         ``local://?`` is stripped to the bare path), rather than stat-ing the raw URI
string."""

```

```

57     sc = _storage_dict(storage_config)
58     uri = resolve_input_uri(
59         raw,
60         input_backend=sc.get("input_backend", "local"),
61         input_root=sc.get("input_root", ""),
62     )
63     return parse_uri(uri)
64
65
66     def input_is_local(raw: str, storage_config: Any = None) -> bool:
67         """True if ``raw`` resolves (under the input config) to a LOCAL filesystem path ?
i.e. a
68         plain on-disk file the caller can stat/validate directly. False for a bucket / Drive
URI
69         that must be fetched first. Raises ``ValueError`` for an unsupported scheme (same as
70         ``resolve_input_ref``)."""
71         return resolve_input_ref(raw, storage_config).backend == "local"
72
73
74     def acquire_local_input(
75         raw: str, storage_config: Any = None, *, scratch_parent: "str | None" = None
76     ) -> str:
77         """Resolve ``raw`` (a path or storage URI) to a LOCAL file path ready for
processing.
78
79         Local / bare paths are returned UNCHANGED (identity passthrough, no copy ? today's
behaviour
80         byte-for-byte). A bucket / Drive source (``gs://``/``s3://``/``gdrive://``, or a
bare name under a
81         non-local ``input_root``/``input_backend``) is downloaded into a fresh scratch dir
and the local
82         path returned. The caller owns successful remote scratch copies; use
83         ``cleanup_acquired_local_input`` when the consumer has finished reading the returned
path."""
84         sc = _storage_dict(storage_config)
85         ref = resolve_input_ref(raw, sc)
86         if ref.backend == "local":
87             return (
88                 ref.key
89             ) # identity ? no temp, no copy, byte-identical to passing the path directly
90         scratch = tempfile.mkdtemp(prefix="rally-input-", dir=scratch_parent)
91         try:
92             return fetch(ref.uri, os.path.join(scratch, ref.name or "in.mp4"), config=sc)
93         except Exception:
94             shutil.rmtree(scratch, ignore_errors=True)
95             raise
96
97
98     def cleanup_acquired_local_input(
99         raw: str, local_path: str, storage_config: Any = None
100     ) -> None:
101         """Delete a remote scratch dir produced by ``acquire_local_input``.
102
103         Local inputs are identity paths and must never be removed. Remote acquisitions are
downloaded
104         into a fresh ``rally-input-`` directory; deleting that directory is safe once the
caller has
105         finished hashing/validating/stitching/segmenting the returned file.
106         """
107         try:
108             if input_is_local(raw, storage_config):
109                 return
110         except ValueError:
111             return
112         if not local_path:

```

```

113         return
114     scratch = os.path.dirname(os.path.abspath(local_path))
115     if os.path.basename(scratch).startswith("rally-input-") and os.path.isdir(scratch):
116         shutil.rmtree(scratch, ignore_errors=True)
117
118
119     def _backend_class(name: str):
120         """Lazy-resolve a backend class so cloud SDKs aren't imported until needed."""
121         if name == "local":
122             from backend.storage.local import LocalStorage
123
124             return LocalStorage
125         if name == "s3":
126             from backend.storage.s3 import S3Storage
127
128             return S3Storage
129         if name == "gcs":
130             from backend.storage.gcs import GCSStorage
131
132             return GCSStorage
133         if name == "gdrive":
134             from backend.storage.gdrive import GDriveStorage
135
136             return GDriveStorage
137         if name == "gdrive-api":
138             from backend.storage.gdrive_api import GDriveApiStorage
139
140             return GDriveApiStorage
141         raise ValueError(
142             f"unknown storage backend: {name!r} (expected local|s3|gcs|gdrive|gdrive-api)"
143         )
144
145
146     def get_storage(
147         backend: Optional[str] = None,
148         *,
149         config: Optional[Dict[str, Any]] = None,
150         **overrides,
151     ) -> StorageBackend:
152         """Construct a backend. ``backend`` (or ``config['backend']``, default 'local')
153         selects the implementation; its per-backend settings come from ``config[backend]`` merged
154         with ``overrides``. Settings carry NO secrets ? only endpoints/regions/paths/source-
155         selectors."""
156         cfg = config or {}
157         name = backend or cfg.get("backend", "local")
158         # Per-backend settings live under the backend name; tolerate the hyphen?underscore
159         # form so a hyphenated backend ("gdrive-api") can read a pydantic field that can't contain a
160         # hyphen
161         # ("gdrive_api"). Hyphen-free names are unaffected (the first lookup wins).
162         settings: Dict[str, Any] = dict(
163             cfg.get(name) or cfg.get(name.replace("-", "_")) or {}
164         )
165         settings.update(overrides)
166         return _backend_class(name)(**settings)
167
168     def fetch(
169         uri: str,
170         dst: "str | os.PathLike[str] | None" = None,
171         *,
172         config: Optional[Dict[str, Any]] = None,
173         overwrite: bool = False,

```

```

173     ) -> str:
174         """Resolve ``uri`` to a local file path. For ``local`` this is an identity
passthrough
175         (no copy); for s3/gcs it downloads to ``dst``."""
176         ref = parse_uri(uri)
177         return get_storage(ref.backend, config=config).fetch_to_local(
178             ref, dst, overwrite=overwrite
179         )
180
181
182     def put(
183         local_path: "str | os.PathLike[str]",
184         uri: str,
185         *,
186         config: Optional[Dict[str, Any]] = None,
187         content_type: Optional[str] = None,
188     ) -> str:
189         """Upload/copy ``local_path`` to ``uri``; return ``uri``."""
190         ref = parse_uri(uri)
191         get_storage(ref.backend, config=config).put(
192             local_path, ref, content_type=content_type
193         )
194         return uri
195
196
197     def list_uris(
198         prefix_uri: str, *, config: Optional[Dict[str, Any]] = None
199     ) -> Iterator[str]:
200         """Yield the URI of every object under ``prefix_uri``."""
201         ref = parse_uri(prefix_uri)
202         for r in get_storage(ref.backend, config=config).list(ref):
203             yield r.uri
204

```

5. user (2026-07-04T08:48:16.357Z)

```

1     """Storage value objects + URI parsing for the pluggable storage layer.
2
3     A ``StorageRef`` is a backend-agnostic pointer to one object. URIs:
4
5         <bare path>                -> local    (today's behaviour; e.g. C:/v/x.mp4,
/data/x.mp4)
6         local:///<path>            -> local    (local:///abs or local://rel/p)
7         s3:///<bucket>/<key>       -> s3        (AWS + R2 + DO Spaces + MinIO via
endpoint_url)
8         gcs:///<bucket>/<key>      -> gcs        (gs:// accepted, normalized to gcs://)
9         gdrive:///<path>          -> gdrive    (path under your locally-MOUNTED Google
Drive "My Drive")
10
11     No cloud SDKs are imported here ? this module is pure stdlib so ``import
backend.storage``
12     stays cheap.
13     """
14
15     from __future__ import annotations
16
17     import re
18     from dataclasses import dataclass
19     from datetime import datetime
20     from typing import Optional
21
22     _SCHEME_RE = re.compile(r"^[A-Za-z][A-Za-z0-9+.\-]*://")
23     _KNOWN = ("local", "s3", "gcs", "gdrive", "gdrive-api")
24
25

```

```

26     @dataclass(frozen=True)
27     class StorageStat:
28         """Lightweight object metadata (uniform across backends)."""
29
30         size: int
31         modified: Optional[datetime] = None
32         etag: Optional[str] = None
33         content_type: Optional[str] = None
34
35
36     @dataclass(frozen=True)
37     class StorageRef:
38         """A backend + a location. ``bucket`` is the S3/GCS bucket; for ``local`` and
39         ``gdrive`` it is
40         empty and ``key`` carries the path (a filesystem path, or a path under the mounted
41         Drive)."""
42
43         backend: str
44         bucket: str
45         key: str
46         uri: str
47
48         @property
49         def name(self) -> str:
50             return self.key.rstrip("/").rsplit("/", 1)[-1]
51
52     def parse_uri(uri: str) -> StorageRef:
53         """Parse a storage URI (or a bare filesystem path) into a ``StorageRef``.
54
55         A string with no ``scheme://`` prefix (including windows ``C:\\...`` paths, which
56         have no ``//``) is treated as a local filesystem path ? preserving today's
57         behaviour.
58         """
59         s = str(uri)
60         m = _SCHEME_RE.match(s)
61         if not m:
62             return StorageRef("local", "", s, f"local://{s}")
63         scheme = m.group(1).lower()
64         rest = s[m.end():]
65         if scheme == "gs":
66             scheme = "gcs"
67         if scheme in ("local", "gdrive", "gdrive-api"):
68             # Path-keyed, no bucket: local FS, a path under the mounted Drive, or a path
69             under a Drive-API
70             # root folder (gdrive-api resolves the path ? a Drive file id).
71             return StorageRef(scheme, "", rest, s)
72         if scheme in _KNOWN:
73             bucket, _, key = rest.partition("/")
74             return StorageRef(scheme, bucket, key, s)
75         raise ValueError(f"unsupported storage URI scheme: {scheme}://{ (in {s!r})")
76
77     def resolve_input_uri(
78         raw: str, *, input_backend: str = "local", input_root: str = ""
79     ) -> str:
80         """Resolve a user-supplied input reference into a storage URI
81         (COMPUTE_DECOUPLED_SERVING M2).
82
83         Precedence (first match wins):
84         1. An explicit ``scheme://`` on ``raw`` (``gs://``, ``s3://``, ``local://``,
85         ``gdrive://``) ? as-is.
86         2. An ABSOLUTE local path (``/data/x.mp4``, ``C:\\?``, ``C:/?``) ? as-is (always
87         local).
88         3. A bare/relative name + a non-empty ``input_root`` ? joined under it

```

```

84         (``gs://bucket/videos`` + ``x.mp4`` ? ``gs://bucket/videos/x.mp4``).
85     4. A bare/relative name + ``input_backend != "local"`` ? prefixed (``gcs`` +
86     ``bucket/x`` ?
87     ``gcs://bucket/x``).
88     5. Otherwise ? ``raw`` unchanged (a relative/bare LOCAL path ? today's behaviour).
89
90     So the default (``input_backend="local"`` , ``input_root=""``) is an identity
91     passthrough for every
92     path used today, preserving byte-for-byte behaviour.
93     """
94     s = str(raw)
95     if _SCHEME_RE.match(s): # 1. explicit scheme wins
96         return s
97     # 2. an anchored LOCAL path is always local, never joined. Detect this OS-
98     INDEPENDENTLY ?
99     # os.path.isabs is platform- AND Python-version-dependent (on Windows + py3.13 a
100     POSIX-style
101     # "/data/x" is NOT absolute). Anchored == a leading "/" or "\\", or a Windows
102     drive prefix "X:".
103     if s.startswith(("/", "\\")) or (len(s) >= 2 and s[1] == ":" and s[0].isalpha()):
104         return s
105     if input_root: # 3. join a relative name under the configured root
106         return input_root.rstrip("/") + "/" + s.lstrip("/")
107     if (
108         input_backend and input_backend != "local"
109     ): # 4. prefix a bucket/key with the declared backend
110         return f"{input_backend}:{s}"
111     return s # 5. bare/relative local path ? unchanged

```

6. assistant (2026-07-04T08:48:37.132Z)

Now let me look at the existing staging tests.

7. user (2026-07-04T08:48:38.683Z)

```

236: def test_cloud_stages_local_upload_to_bucket(cloud_env, tmp_path, monkeypatch):
237:     """A browser upload lands on THIS box's local disk; under cloud-serving with a CLOUD
238:     input_root it
239:     is STAGED to the bucket and the L4 job dispatches from the staged gs:// URI (not hard-
240:     failed) ? #461.
241:     The DB video record is repointed at the staged source so a later /api/compile can re-
242:     fetch it."""
243:     m.global_config.storage.input_root = "gs://in-bucket/videos"
244:     # 1) the local upload was staged UP to the input bucket, namespaced by video_id
245:     # etc. survives) ? not a fresh default client. The model_dump carries
246:     input_root/input_backend.
247:     assert put_cfg is not None and put_cfg.get("input_root") == "gs://in-bucket/videos"
248:     # 3) the DB video record now points at the staged source (so compile re-fetches it)
249: def test_cloud_local_upload_without_input_root_still_fails(cloud_env, tmp_path,
250: monkeypatch):
251:     """No cloud input_root configured ? we genuinely can't stage a local upload ? hard-fail
252:     (no
253:     _db, _bucket, _mp = cloud_env # fixture leaves input_root unset (input_backend=local)
254:     assert fake.calls == [] and called == [] # neither staged nor dispatched

```

8. user (2026-07-04T08:48:43.790Z)

```

1     """API cloud-serving dispatch (COMPUTE_DECOUPLED_SERVING M6 ? the API path).
2
3     Proves the /api/process job path can offload DETECT to a Cloud Run GPU Job and ingest
4     the rally
5     intervals back into the API DB, so /api/query + /api/compile (local stitch) are
6     unchanged ? all

```

```

5      GPU-free + cloud-free via a FAKE CloudRunJobClient that writes report.json +
intervals.csv + the
6      done-marker into a local "bucket". Cardinal invariant: DEFAULT-OFF ? only
compute_target=cloud_run
7      dispatches; every other target runs the in-process segmenter byte-identically.
8      """
9
10     import csv
11     import json
12     import os
13
14     import pytest
15
16     pytest.importorskip("httpx")
17     from fastapi.testclient import TestClient
18
19     import backend.compute as compute_pkg
20     import backend.main as m
21     from backend.compute.cloud_run import CloudRunJobClient
22     from backend.config.models import AppConfig
23     from backend.infrastructure.database import Database
24     from backend.infrastructure.execution_policy import ExecutionPolicy
25     from backend.pipeline.validators.base import ValidationResult
26
27     _FAST = ExecutionPolicy(poll_every_n_sec=0.0, op_timeout_sec=5.0, max_wait_sec=5.0)
28
29
30     def _pass():
31         return [ValidationResult(validator_name="x", passed=True, message="ok", details={})]
32
33
34     class _InlineExecutor:
35         def submit(self, fn, *a, **k):
36             fn(*a, **k)
37
38
39     class _FakeCloudClient(CloudRunJobClient):
40         """Simulates the Cloud Run worker: on dispatch, write report.json + intervals.csv +
the done-marker
41         into the local bucket exactly where the real worker would, so the bucket-poll +
ingest resolve."""
42
43         def __init__(self, *, video_id="vidCloud", returncode=0, rows=None):
44             self.calls: list[list[str]] = []
45             self.video_id, self.returncode = video_id, returncode
46             self.rows = (
47                 rows
48                 if rows is not None
49                 else [(2.74, 6.91, 5, "winner"), (8.74, 11.38, 3, "unforced_error")]
50             )
51
52         def run_job(self, job_name, argv, *, region, project=None):
53             self.calls.append(list(argv))
54             out_uri = argv[argv.index("--out-uri") + 1].rstrip("/")
55             done_uri = argv[argv.index("--done-uri") + 1]
56             if self.returncode == 0:
57                 outputs = os.path.join(out_uri, "outputs", self.video_id)
58                 os.makedirs(outputs, exist_ok=True)
59                 with open(os.path.join(outputs, "report.json"), "w", encoding="utf-8") as f:
60                     json.dump(
61                         {
62                             "video_id": self.video_id,
63                             "segment_count": len(self.rows),
64                             "status": "success",
65

```

```

66         f,
67     )
68     with open(
69         os.path.join(outputs, "intervals.csv"),
70         "w",
71         newline="",
72         encoding="utf-8",
73     ) as f:
74         w = csv.writer(f)
75         w.writerow(["start", "end", "shot_count", "ending_reason"])
76         for s, e, sc, er in self.rows:
77             w.writerow([f"{s:.3f}", f"{e:.3f}", sc, er])
78     os.makedirs(os.path.dirname(done_uri), exist_ok=True)
79     with open(done_uri, "w", encoding="utf-8") as f:
80         json.dump(
81             {
82                 "video_id": self.video_id,
83                 "returncode": self.returncode,
84                 "segment_count": len(self.rows),
85                 "status": "success",
86             },
87             f,
88         )
89     return "exec-1"
90
91     def cancel_execution(self, execution):
92         raise AssertionError(
93             "cancel must never fire on the happy path"
94         ) # pragma: no cover
95
96
97     @pytest.fixture
98     def cloud_env(tmp_path, monkeypatch):
99         """API wired for cloud_run with a LOCAL bucket (the local storage backend makes the
100         bucket read an
101         identity file read ? no GCS). A gs:// input is 'fetched' to a local fake file for
102         hash+validate."""
103         db = Database(str(tmp_path / "t.db")) # noqa: F841
104         monkeypatch.setattr(m, "db_instance", db)
105         monkeypatch.setattr(m, "db_instance", db)
106         bucket = tmp_path / "bucket"
107         bucket.mkdir()
108         cfg = AppConfig.model_validate(
109             { # noqa: F841
110                 "deployment": {"compute_target": "cloud_run"},
111                 "storage": {"backend": "local", "output_root": str(bucket)},
112             }
113         )
114         monkeypatch.setattr(m, "global_config", cfg)
115         monkeypatch.setattr(m, "global_config", cfg)
116         monkeypatch.setattr(m, "_get_executor", lambda: _InlineExecutor())
117         # A real gs:// input would be fetched to hash+validate; return a local fake so that
118         path is offline.
119         fake_local = tmp_path / "fetched.mp4"
120         fake_local.write_bytes(b"FAKEVIDEOBYTES")
121         monkeypatch.setattr(
122             m.storage, "acquire_local_input", lambda path, scfg: str(fake_local)
123         )
124         monkeypatch.setattr(
125             m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {})
126         )
127         return db, bucket, monkeypatch
128
129     def _inject_fake(monkeypatch, fake):

```

```

128         """Make backend.compute.get_compute_backend (re-imported inside
129         _maybe_dispatch_remote) build a
130         real CloudRunCompute around the FAKE client + the fast no-wait policy + a fixed
131         token. ``**kw``
132         forwards the per-request ``target_override`` (item 3) so the cloud-picker path
133         resolves too."""
134         real = compute_pkg.get_compute_backend
135         monkeypatch.setattr(
136             compute_pkg,
137             "get_compute_backend",
138             lambda config, **kw: real(
139                 config, run_client=fake, policy=_FAST, token="tok", **kw
140             ),
141         )
142
143     def _meta(row):
144         md = row.get("metadata")
145         return json.loads(md) if isinstance(md, str) else (md or {})
146
147     def test_cloud_dispatch_ingests_segments(cloud_env):
148         db, _bucket, monkeypatch = cloud_env
149         fake = _FakeCloudClient()
150         _inject_fake(monkeypatch, fake)
151
152         out = m._execute_processing(
153             m.ProcessRequest(video_path="gs://b/clip.mp4", segmenter="wasb_hybrid")
154         )
155
156         assert out["status"] == "success"
157         assert len(out["play_segments"]) == 2
158         rows = db.query_play_segments(out["video_id"], {})
159         assert len(rows) == 2
160         metas = [_meta(r) for r in rows]
161         assert all(md["source"] == "cloud_run" for md in metas)
162         assert {md.get("shot_count") for md in metas} == {5, 3}
163         assert {md.get("ending_reason") for md in metas} == {"winner", "unforced_error"}
164
165     def test_cloud_dispatch_builds_correct_spec(cloud_env):
166         _db, bucket, monkeypatch = cloud_env
167         fake = _FakeCloudClient()
168         _inject_fake(monkeypatch, fake)
169
170         m._execute_processing(
171             m.ProcessRequest(video_path="gs://b/clip.mp4", segmenter="wasb_hybrid")
172         )
173
174         assert len(fake.calls) == 1 # exactly one Cloud Run Job execution
175         argv = fake.calls[0]
176         assert argv[argv.index("--video-uri") + 1] == "gs://b/clip.mp4"
177         assert argv[argv.index("--out-uri") + 1] == str(bucket)
178         assert argv[argv.index("--segmenter") + 1] == "wasb_hybrid"
179         joined = " ".join(argv)
180         assert (
181             "deployment.compute_target=local_gpu" in joined
182         ) # container runs INLINE (never re-dispatch)
183         assert "indexing.detector_impl=native" in joined
184         assert (
185             "indexing.skip_ai_handoff=false" in joined
186         ) # the engine's AI-handoff stance is forwarded
187
188     def test_cloud_dispatch_forwards_skip_ai_handoff_true(cloud_env):

```

```

190         """When the engine runs no-AI (skip_ai_handoff=True), the cloud job must too ? else
it tries the
191         Gemini handoff (which the cloud job has no key for) and yields 0 rallies. The flag
is forwarded."""
192         _db, _bucket, monkeypatch = cloud_env
193         m.global_config.indexing.skip_ai_handoff = True
194         fake = _FakeCloudClient()
195         _inject_fake(monkeypatch, fake)
196         m._execute_processing(
197             m.ProcessRequest(video_path="gs://b/clip.mp4", segmenter="wasb_hybrid")
198         )
199         assert "indexing.skip_ai_handoff=true" in " ".join(fake.calls[0])
200
201
202     def test_cloud_failure_marks_failed_no_rows(cloud_env):
203         db, _bucket, monkeypatch = cloud_env
204         # pre-seed a prior good row ? must be preserved on a cloud failure (destroy-AFTER-
confirm).
205         db.add_video("vidLocalPrior", "gs://b/clip.mp4", "processed", [])
206         fake = _FakeCloudClient(returncode=1)
207         _inject_fake(monkeypatch, fake)
208
209         out = m._execute_processing(
210             m.ProcessRequest(video_path="gs://b/clip.mp4", segmenter="wasb_hybrid")
211         )
212         assert out["status"] == "failed"
213         assert "returncode 1" in out["message"]
214         assert out["play_segments"] == []
215         assert db.query_play_segments(out["video_id"], {}) == []
216         # It DID dispatch + run remotely (then failed rc=1), so the path is honestly
cloud_run + dispatched.
217         assert (
218             out["compute"]["path"] == "cloud_run" and out["compute"]["dispatched"] is True
219         )
220
221
222     def test_cloud_requires_a_bucket_input(cloud_env, tmp_path):
223         _db, _bucket, monkeypatch = cloud_env
224         fake = _FakeCloudClient()
225         _inject_fake(monkeypatch, fake)
226         local_path = str(tmp_path / "local_only.mp4")
227
228         out = m._execute_processing(
229             m.ProcessRequest(video_path=local_path, segmenter="wasb_hybrid")
230         )
231         assert out["status"] == "failed"
232         assert "cloud input" in out["message"]
233         assert fake.calls == [] # never dispatched a local-only path
234
235
236     def test_cloud_stages_local_upload_to_bucket(cloud_env, tmp_path, monkeypatch):
237         """A browser upload lands on THIS box's local disk; under cloud-serving with a CLOUD
input_root it
238         is STAGED to the bucket and the L4 job dispatches from the staged gs:// URI (not
hard-failed) ? #461.
239         The DB video record is repointed at the staged source so a later /api/compile can
re-fetch it."""
240         db, _bucket, _mp = cloud_env
241         m.global_config.storage.input_backend = "gcs"
242         m.global_config.storage.input_root = "gs://in-bucket/videos"
243         puts: list = []
244
245         def _fake_put(src, uri, **kw):
246             puts.append((str(src), uri, kw.get("config")))
247             return uri

```

```

248
249 monkeypatch.setattr(m.storage, "put", _fake_put)
250 fake = _FakeCloudClient()
251 _inject_fake(monkeypatch, fake)
252 local_path = str(tmp_path / "upload.mp4")
253
254 out = m._execute_processing(
255     m.ProcessRequest(video_path=local_path, segmenter="wasb_hybrid")
256 )
257
258 assert out["status"] == "success"
259 # 1) the local upload was staged UP to the input bucket, namespaced by video_id
260 assert len(puts) == 1
261 src, dest, put_cfg = puts[0]
262 assert src == local_path
263 assert dest.startswith("gs://in-bucket/videos/") and dest.endswith("/upload.mp4")
264 # the active storage settings are threaded into put() (so a non-default S3 endpoint
/ GCS project /
265 # etc. survives) ? not a fresh default client. The model_dump carries
input_root/input_backend.
266 assert put_cfg is not None and put_cfg.get("input_root") == "gs://in-bucket/videos"
267 assert put_cfg.get("input_backend") == "gcs"
268 # 2) the L4 job dispatched from the STAGED gs:// URI, not the unreadable local path
269 assert len(fake.calls) == 1
270 argv = fake.calls[0]
271 assert argv[argv.index("--video-uri") + 1] == dest
272 # 3) the DB video record now points at the staged source (so compile re-fetches it)
273 row = db.get_video(out["video_id"])
274 assert row is not None and row["filepath"] == dest
275
276
277 def test_cloud_local_upload_without_input_root_still_fails(cloud_env, tmp_path,
monkeypatch):
278     """No cloud input_root configured ? we genuinely can't stage a local upload ? hard-
fail (no
279     dispatch), rather than dispatching a path the Cloud Run job can't read."""
280     _db, _bucket, _mp = cloud_env # fixture leaves input_root unset
(input_backend=local)
281     fake = _FakeCloudClient()
282     _inject_fake(monkeypatch, fake)
283     called = []
284     monkeypatch.setattr(m.storage, "put", lambda *a, **k: called.append(a))
285
286     out = m._execute_processing(
287         m.ProcessRequest(video_path=str(tmp_path / "up.mp4"), segmenter="wasb_hybrid")
288     )
289     assert out["status"] == "failed"
290     assert "cloud input" in out["message"]
291     assert fake.calls == [] and called == [] # neither staged nor dispatched
292
293
294 def test_default_off_runs_in_process(tmp_path, monkeypatch):
295     """compute_target='' ? get_compute_backend returns None ? the in-process segmenter
runs
296     byte-identically (the gate is inert). No Cloud Run dispatch."""
297     db = Database(str(tmp_path / "t.db")) # noqa: F841
298     monkeypatch.setattr(m, "db_instance", db)
299     monkeypatch.setattr(m, "db_instance", db)
300     cfg = AppConfig() # compute_target defaults to "" # noqa: F841

```

9. assistant (2026-07-04T08:48:53.704Z)

Let me check the StorageConfig model.

10. user (2026-07-04T08:48:55.402Z)

```

399-     backend: Literal["local", "s3", "gcs", "gdrive", "gdrive-api"] = "local"
400-     # Output root for produced artifacts. "" => fall back to output.output_dir (non-
breaking).
401-     output_root: str = ""
402-     # --- Input side (COMPUTE_DECOUPLED_SERVING M2; parallel to the output
`backend`/`output_root`).
403-     # Default ``local`` \ "" keeps today's behaviour: a bare path / ``local://`` URI is read
IN PLACE
404-     # (identity, no copy). Set these to read inputs from a bucket / mounted Drive ? a
bare/relative
405-     # input name is resolved against ``input_root`` (e.g.
``input_root="gs://bucket/videos"`` + "x.mp4"
406-     # => ``gs://bucket/videos/x.mp4``) or, if ``input_root`` is empty, prefixed with
``input_backend``
407-     # (``"gcs"`` + "bucket\x.mp4" => ``gcs://bucket/x.mp4``). An explicit scheme on the
input
408-     # (``gs://``/``s3://``) or an absolute local path ALWAYS wins. Non-local inputs are
fetched to a
409-     # scratch dir before processing (see ``backend.storage.acquire_local_input``).
410-     input_backend: Literal["local", "s3", "gcs", "gdrive", "gdrive-api"] = "local"
411-     input_root: str = ""
412-     # --- Per-video workspace (docs/PER_VIDEO_WORKSPACE.md) ---
413-     # One folder per video keyed by the MD5 video_id:
<root>/[<workspace_prefix>]/<video_id>/...
414-     # so a local deploy and a cloud (bucket) deploy share ONE relative layout (only the root
415-     # differs). DEFAULT-OFF: convergence is phased + flipped via a Launch Report (D5). When
OFF
416-     # the legacy flat layout is used unchanged.
417-     single_folder_per_video: bool = False

```

11. user (2026-07-04T08:49:19.967Z)

```

219:# dispatch+ingest, compile resolution + worker handler) now lives in backend.orchestration
221:# backend.main ? `import backend.main as m; m._execute_processing(...)` the job worker's
224:from backend.orchestration import ( # noqa: F401 (re-exports: names stay addressable on
backend.main)
228:     _execute_processing,

```

12. user (2026-07-04T08:49:20.981Z)

```

21:import csv
22:import json
23:import logging
24:import math
25:import os
26:import tempfile
27:from typing import TYPE_CHECKING, Any, Dict, List, Optional, Tuple
28:from urllib.parse import quote
29:from backend import storage
30:from backend.config import AppConfig, StitchingConfig
31:from backend.pipeline.segmenters.base import segmenter_registry
32:from backend.pipeline.stitcher.watermark_i18n import localize_watermark
33:from backend.pipeline.validators.base import registry
34:from backend.results import Failure
35:from backend.utils.paths import output_base, safe_output_filename
167:def _maybe_dispatch_remote(

```

13. assistant (2026-07-04T08:49:24.847Z)

Now I'll apply the fix.

14. user (2026-07-04T08:49:32.379Z)

The file C:/Users/avidu/Projects/khelsutra-guru/wt-upload-staging/backend/orchestration.py has

been updated successfully. (file state is current in your context ? no need to Read it back)

15. user (2026-07-04T08:49:48.419Z)

The file C:/Users/avidu/Projects/khelsutra-guru/wt-upload-staging/tests/test_api_cloud_dispatch.py has been updated successfully. (file state is current in your context ? no need to Read it back)

16. assistant (2026-07-04T08:49:52.246Z)

Quick targeted check before the full suite.

17. user (2026-07-04T08:50:11.373Z)

```
..... [100%]
===== warnings summary =====
tests/test_api_cloud_dispatch.py::test_full_api_process_to_query_via_cloud
tests/test_api_cloud_dispatch.py::test_local_override_survives_api_boundary
tests/test_api_cloud_dispatch.py::test_cloud_override_survives_api_boundary_on_local_server
tests/test_api_cloud_dispatch.py::test_full_api_process_job_result_carries_cloud_provenance
  C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\site-
packages\google\auth\_default.py:113: UserWarning: Your application has authenticated using end
user credentials from Google Cloud SDK without a quota project. You might receive a "quota
exceeded" or "API not enabled" error. See the following page for troubleshooting:
https://cloud.google.com/docs/authentication/adc-troubleshooting/user-creds.
    warnings.warn(_CLOUD_SDK_CREDENTIALS_WARNING)

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
26 passed, 4 warnings in 15.38s
```

18. assistant (2026-07-04T08:50:19.152Z)

The new regression test passes. Now the three gates ? lint and types first, then the full suite.

19. user (2026-07-04T08:50:21.171Z)

ruff 0.15.16
All checks passed!

20. user (2026-07-04T08:50:26.308Z)

```
mypy 2.1.0 (compiled: yes)
backend\annotations\base.py:88: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs [annotation-unchecked]
backend\sports\__init__.py:9: note: By default the bodies of untyped functions are not checked,
consider using --check-untyped-defs [annotation-unchecked]
backend\pipeline\validators\base.py:44: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs [annotation-unchecked]
backend\providers\__init__.py:9: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs [annotation-unchecked]
Success: no issues found in 147 source files
```

21. assistant (2026-07-04T08:50:30.462Z)

Lint and types are green. Now the full suite.

22. user (2026-07-04T08:50:32.926Z)

Command running in background with ID: bv1kfcwbl. Output is being written to:
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\d4cf3fd6-b5bd-4ed1-9b57-88eea69333cd\tasks\bv1kfcwbl.output. You will be notified when
it completes. To check interim output, use Read on that file path.

23. assistant (2026-07-04T08:50:39.043Z)

While the suite runs, let me verify the diff contains only my intended changes and double-check the reviewer's exact repro shape against the new gate logic.

24. user (2026-07-04T08:50:42.184Z)

```
M backend/orchestration.py
M tests/test_api_cloud_dispatch.py
fix/upload-gcs-staging
d3a9629 review: thread active storage config into the staging put (#461)
```

25. user (2026-07-04T08:50:43.685Z)

```
gcs
local
s3
```

26. user (2026-07-04T08:51:00.379Z)

```
1 .....SS.....
```

27. assistant (2026-07-04T08:51:08.148Z)

The suite is running (48 tests in so far, 2 skips). I'll wait for the background task to complete ? it notifies on exit.

28. user (2026-07-04T08:51:11.256Z)

Monitor started (task bu39bwodm, timeout 600000ms). You will be notified on each event. Keep working ? do not poll or sleep. Events may arrive while you are waiting for the user ? an event is not their reply.

29. user (2026-07-04T08:51:17.345Z)

```
250         if input_ref.backend == "local":
251             # A browser upload lands on THIS box's local disk; the Cloud Run job (a
different machine)
252             # can't read it. Stage it UP to the input bucket first, then dispatch from there
(#461). Only
253             # possible when a CLOUD input_root is configured; without one we genuinely can't
stage ? fail.
254             # Classify the ROOT ITSELF (parse_uri), not storage.input_backend: the staged
URI below is
255             # built by joining under input_root, and storage.put routes by
parse_uri(staged_uri).backend ?
256             # so an explicit cloud root (gs://?, s3://?) stages fine with input_backend left
at its
257             # default, while a bare/local-looking root can't stage no matter what
input_backend says
258             # (resolve_input_uri only consults input_backend when input_root is EMPTY, and
an empty root
259             # gives us nothing to build the staged URI under).
260             input_root = (getattr(storage_cfg, "input_root", "") or "").strip()
261             try:
262                 root_backend = (
263                     storage.parse_uri(input_root).backend if input_root else "local"
264                 )
265             except ValueError: # unsupported scheme in input_root ? nowhere real to stage
to
266                 root_backend = "local"
267             if root_backend == "local":
268                 return _failed(
269                     "cloud-serving needs a cloud input (e.g. gs://?), or a cloud
```

```

storage.input_root to "
270             "stage local uploads into; a path local to this machine can't be read by
the Cloud "
271             "Run job.",
272             False,
273         )
274         local_src = input_ref.key # resolved local path of the upload
275         staged_uri =
f"{input_root.rstrip('/')}/{video_id}/{os.path.basename(local_src)}"
276         try:
277             notify("staging upload to bucket")
278             # Thread the active storage settings so backend-specific config survives (S3
endpoint_url/
279             # region/addressing for R2·D0·MinIO; GCS project/client) ? not a fresh
default client.

```

30. user (2026-07-04T08:51:23.770Z)

```

#1 [in_progress] Fix #468 second blocker: claim_due_job false-empty on lost CAS race
#2 [in_progress] Sweep all 4 open PRs (#465/#467/#468/#469) for new review comments and address
them
#3 [pending] Run full e2e deploy test after all 4 PRs merge
#4 [pending] Capture rough edges: fix small ones, file GitHub issues for the rest

```

31. assistant (2026-07-04T08:51:29.138Z)

Waiting for the full-suite result before reporting.

32. user (2026-07-04T08:51:33.165Z)

```

1      .....SS..... [ 4%]
2      .....

```

33. user (2026-07-04T08:51:43.117Z)

Command running in background with ID: bgan6m1p1. Output is being written to:
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\d4cf3fd6-b5bd-4ed1-9b57-88eea69333cd\tasks\bgan6m1p1.output. You will be notified when
it completes. To check interim output, use Read on that file path.

34. user (2026-07-04T08:51:46.982Z)

```

{"message":"Successfully stopped task: bu39bwodm (while kill -0 $(cat /dev/null) 2>/dev/null; do
:; done; f=\"C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-
highlight-indexer/d4cf3fd6-b5bd-4ed1-9b57-88eea69333cd/tasks/bv1kfcwbl.output\"; until grep -qE
\"passed|failed|error\" \"${f}\" 2>/dev/null; do sleep 5; done; tail -c 600
\"${f}\",\"task_id\":\"bu39bwodm\",\"task_type\":\"local_bash\",\"command\":\"while kill -0 $(cat
/dev/null) 2>/dev/null; do :; done; f=\"C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-
Projects-badminton-highlight-
indexer/d4cf3fd6-b5bd-4ed1-9b57-88eea69333cd/tasks/bv1kfcwbl.output\"; until grep -qE
\"passed|failed|error\" \"${f}\" 2>/dev/null; do sleep 5; done; tail -c 600 \"${f}\""}

```

35. assistant (2026-07-04T08:51:52.169Z)

Waiting for the suite to finish (both the pytest task and the waiter will notify).

36. user (2026-07-04T08:51:53.265Z)

```

1      .....SS..... [ 4%]
2      .....S..... [ 8%]
3      ..... [ 13%]
4      ..... [ 17%]
5      ..... [ 21%]
6      .....

```

37. assistant (2026-07-04T08:52:00.318Z)

Suite progressing cleanly at ~24% with no failures. I'll wait for the background completion notification before reporting final results.

38. user (2026-07-04T08:52:00.416Z)

[structured-output-enforce] You MUST call the StructuredOutput tool to complete this request. Call this tool now.

39. user (2026-07-04T08:52:08.330Z)

```
1      .....SS..... [ 4%]
2      .....S..... [ 8%]
3      ..... [ 13%]
4      ..... [ 17%]
5      ..... [ 21%]
6      ..... [ 26%]
7      ..... [ 30%]
8      ..... [ 35%]
9      ..... [ 39%]
10     ..... [ 43%]
11     ..... [ 48%]
12     .....
```

40. assistant (2026-07-04T08:52:16.366Z)

The suite is at ~50%. I need the final gate result before reporting, so let me block on it synchronously.

41. user (2026-07-04T08:52:49.653Z)

```
..... [100%]
===== warnings summary =====
tests/test_api_cloud_dispatch.py::test_full_api_process_to_query_via_cloud
tests/test_api_cloud_dispatch.py::test_local_override_survives_api_boundary
tests/test_api_cloud_dispatch.py::test_cloud_override_survives_api_boundary_on_local_server
tests/test_api_cloud_dispatch.py::test_full_api_process_job_result_carries_cloud_provenance
tests/test_api_endpoints.py::test_process_nonlocal_uri_skips_exists_and_fetches
tests/test_api_endpoints.py::test_process_remote_input_reports_use_durable_output_dir
tests/test_api_endpoints.py::test_process_bucket_uri_yields_queryable_video_id[gs://bucket/video
s/x.mp4]
tests/test_api_endpoints.py::test_process_bucket_uri_yields_queryable_video_id[gs://bucket/vide
os/x.mp4]
  C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\site-
packages\google\auth\_default.py:113: UserWarning: Your application has authenticated using end
user credentials from Google Cloud SDK without a quota project. You might receive a "quota
exceeded" or "API not enabled" error. See the following page for troubleshooting:
https://cloud.google.com/docs/authentication/adc-troubleshooting/user-creds.
    warnings.warn(_CLOUD_SDK_CREDENTIALS_WARNING)

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
1629 passed, 10 skipped, 8 warnings in 128.78s (0:02:08)
```

42. user (2026-07-04T08:53:12.109Z)

Structured output provided successfully